Keep the Ledger, Drop the Logs: State-Keyed Context Compaction for Long-Horizon LLM Agents

Anonymous authors
Paper under double-blind review

Abstract

Long-horizon LLM agents must compress their growing interaction histories, and most compressors treat the history as text to be shortened—by recency, masking, salience or summarization. We argue that an agent's history is better viewed as an *event log that mutates a state*, and that what compression must preserve is the *current value of each state variable*. This view exposes two failure modes: *missing* state, and the more insidious *stale* state, where the newest value is dropped but an older one survives. We formalize compression with keyed state writes and an ideal reader and prove that the compacted log (the latest record per key) is a sufficient statistic whose size is independent of the horizon; that supersession-consistent compressors are never stale while recency windows forget keys by age; and that supersession-blind selection is stale with probability $(1-\rho)(1-(1-\rho)^{m-1})$ for a key with m writes, a law that also governs imperfect extraction. We propose State-Keyed Compaction (SKC), a model-free compressor with three layers: a verbatim latest-value ledger, supersession-aware deduplication, and a knapsack scored by a utility model trained for free from logged trajectories, with redundancy-aware selection that charges each item only for content not yet in the context. Across 4,979 public trajectories from two agent scaffolds and twelve LLM configurations, 65–89% of context tokens are tool outputs and 23–55% are repeated lines. On 2,479 held-out trajectories, SKC keeps 9–13 points more state than the best of seven baselines in-domain (e.g. 67.1% vs. 57.8% at a 4.7× compression) and 7–10 points more on unseen scaffolds and LLMs, with stale rates (0.1–0.2%) as low as a recency window, while preserving as many of the identifiers the next action uses as the best lexical selector in-domain. With three LLM readers from two families, SKC (without redundancy-aware selection) raises the accuracy of questions about the agent's current state from 25–40% to 50–77% ($p < 10^{-3}$) without degrading next-action prediction, while a budget-matched LLM summary with recent turns answers 18–38%. Fine-grained lexical selection still preserves slightly more of the code that agents re-type (1–5 points); an oracle utility shows that this gap is one of prediction, which our framework can absorb.

1 Introduction

Large language model (LLM) agents now routinely run for tens to hundreds of steps: a software-engineering agent opens files, edits them, runs tests, reads tracebacks, and edits again (Yang et al., 2024; Wang et al., 2025). Every step appends an action and an observation to the context. In the 4,979 public trajectories we study (two agent scaffolds, twelve LLM configurations), **65–89% of all context tokens are tool outputs**, a quarter to a half of all tokens are lines that recur later in the same history, and recent models reach back more than ten steps for 60–81% of the identifiers their actions use (Section 7.1). Even when a model's window is large enough, long contexts cost latency and money and degrade accuracy (Liu et al., 2024; Hsieh et al., 2024; Hong et al., 2025). Every practical agent therefore *compresses* its history—by truncating it, masking old observations, pruning "unimportant" tokens, or summarizing it with an LLM (Packer et al., 2023; Lindenbauer et al., 2025; Kang et al., 2025; Anthropic, 2025).

Most of these methods treat the history as *text to be shortened*. We argue that for agents it is better viewed as an *event log that mutates a state*. An edit changes a region of a file; re-running a test replaces the previous result; a user's correction overrides an earlier constraint. What the agent must not lose is the *current value of each state variable*, and a compressor can fail in two distinct ways: it can drop the value (*missing*), or—worse—drop the latest value while keeping an older one, so that a capable reader confidently acts on an outdated state (*stale*). Text-centric criteria such as recency or relevance do not see this distinction: a relevance scorer has no reason to prefer the newer of two test results, and a recency window has no reason to keep a constraint stated fifty steps ago.

This view suggests borrowing a mechanism that databases have used for decades: *log compaction*, which retains only the latest record per key (Kreps et al., 2011; Kleppmann, 2017). We make the idea precise and test it.

Contributions.

1. **A state-centric formalization** of agent context compression (Section 3): histories as event logs with keyed writes, an *ideal reader* that separates information loss from reader ability, three outcomes—*correct*, *stale*, *missing*—for each state probe, and two key-free, behavior-grounded probes (identifiers and verbatim lines that the agent's next action uses).

2. **Theory** (Section 4): the compacted log is a sufficient statistic whose size is independent of the horizon and matches an information-theoretic lower bound; *supersession-consistent* compressors can never be stale; recency windows lose keys by age rather than importance; supersession-blind selection is stale with probability $(1-\rho)(1-(1-\rho)^{m-1})$—a law that also governs a ledger fed by an extractor of recall ρ; a ledger makes any further selection safe; and supersession-aware deduplication never hurts state retention.

3. **State-Keyed Compaction (SKC)** (Section 5): a model-free compressor with three layers—a verbatim latest-value-per-key ledger, supersession-aware deduplication of repeated lines, and a knapsack over ledger records and history chunks scored by a small utility model that is trained *for free* from logged trajectories and transfers across LLMs and agent scaffolds, with redundancy-aware selection that never pays twice for the same line of a turn.

4. **An extensive empirical study**: an ideal-reader evaluation on 2,479 held-out trajectories (SWE-agent on SWE-bench Lite with five LLMs; OpenHands and SWE-agent on SWE-bench Verified with seven further LLM configurations, including open-weight models) at 4k–32k-token budgets, with bootstrap confidence intervals; an LLM-in-the-loop evaluation with three reader models from two families; ablations, an oracle upper bound, and a controllable benchmark that confirms each theoretical prediction.

Main findings. At a 4k-token budget (a 4.7× compression of the average history), SKC keeps the current value of 67% of all state variables, +9.3 points (95% CI $[8.8, 9.8]$) over the best of seven baselines, and the gap grows to +13.4 at 16k; on unseen scaffolds and LLMs it is +6.7 to +10.2. Selection methods leave stale state on 1.0–2.8% of probes (13–18% of the values they keep for keys rewritten three or more times); SKC on 0.1–0.2%, as low as a recency window while retaining far more. With real LLM readers, SKC raises state-question accuracy from 25–40% (best baseline) to 50–77% across three models ($p < 10^{-3}$), while next-action prediction is statistically indistinguishable from the baselines; a budget-matched LLM summary (summary plus recent turns, as in production agents) answers 18–38%. In-domain SKC also preserves the most identifiers that the next action uses, but it is not uniformly best: fine-grained BM25 selection preserves more lines that the agent later copies (by 1–5 points in-domain), which an oracle utility shows is a gap of prediction rather than of the framework.

Scope. Our main evaluation measures *information sufficiency* with an ideal reader, which isolates the compressor from the reader; an LLM-in-the-loop study with three reader models checks that the preserved information is usable, and an ALFWorld study runs an agent end-to-end on compressed contexts (Appendix E); end-to-end runs on SWE-bench remain future work (Section 8). Code, parsed probes, trained utility models and all results are released.

2 Related Work

Prompt and context compression. Token-level pruning methods drop tokens with low self-information or low importance under a small LM, e.g. Selective Context (Li et al., 2023) and the LLMLingua family (Jiang et al., 2023; 2024; Pan et al., 2024). Learned compressors map context into soft tokens (Mu et al., 2023; Chevalier et al., 2023; Ge et al., 2024), and KV-cache methods evict entries by attention statistics or position (Zhang et al., 2023; Xiao et al., 2024). These methods score text by salience; they are not designed around supersession, which our analysis shows is what produces stale state. Our BM25 baseline (Robertson & Zaragoza, 2009) is a turn-level, model-free representative of salience-based selection.

Memory for LLM agents. MemGPT pages information between a bounded context and external storage (Packer et al., 2023); Generative Agents keep a memory stream with recency/importance/relevance retrieval and periodic reflection (Park et al., 2023); Reflexion stores verbal lessons (Shinn et al., 2023). MemoryBank, A-MEM and Mem0 organize long-term memories, and Mem0 explicitly applies add/update/delete operations to stored facts (Zhong et al., 2024; Xu et al., 2025; Chhikara et al., 2025). Recursive or gist summarization compresses long documents and dialogues (Wang et al., 2023; Lee et al., 2024; Chen et al., 2023). Benchmarks such as LoCoMo and LongMemEval test long-term conversational memory including knowledge updates (Maharana et al., 2024; Wu et al., 2025). Our work is complementary: rather than proposing another memory architecture, we give a formal account of *what* must be preserved within a single long-horizon episode, why common compressors fail, and an evaluation that separates missing from stale state.

Context management for long-horizon agents. Closest to us, Lindenbauer et al. (2025) show that simply masking old observations in SWE-agent matches LLM summarization in solve rate at lower cost, and ACON (Kang et al., 2025) optimizes compression guidelines for history and observations with failure-driven feedback. MEM1 trains agents to maintain a compact internal state that replaces the history (Zhou et al., 2025). Practitioner guidance recommends "compaction" and structured note-taking for agents (Anthropic, 2025). We provide the missing analytical lens: observation masking works because agent turns carry most state-changing information and because it is (nearly) supersession-consistent; it fails on state that only lives in observations. SKC can be seen as making the implicit state of these approaches explicit and keyed.

State tracking and logs. Language models' ability to track entity states through a sequence of operations is studied by Kim & Schuster (2023). Event sourcing and log compaction in data systems keep only the latest record per key so that the state can be rebuilt from a bounded log (Kreps et al., 2011; Kleppmann, 2017); streaming lower bounds (Alon et al., 1999) motivate our counting argument.

3 Problem Formulation

Histories and state. An agent history is a sequence of turns $h_{1:t} = (x_1, \ldots, x_t)$, each a piece of text of $\ell(x_i)$ tokens with a role (task, agent, tool, user). Let $\mathcal{K}$ be a set of *state keys* and $\mathcal{V}$ a set of values. Each turn carries a (possibly empty) set of *writes* $\mathrm{W}(x_i) \subseteq \mathcal{K} \times \mathcal{V}$: evidence that key k took value v at time i. The state at time t is the *fold* of the log under last-writer-wins,

$$S_t(k) = v \quad \text{where } (k, v) \in \mathrm{W}(x_i),\ i = \max\{j \le t : k \text{ written in } x_j\}, \tag{1}$$

and $S_t(k) = \perp$ if k was never written. For a software agent, keys include the content of each edited file region, the latest output of each command, the latest view of each file window, the results of each code search, and the task itself (Section 6).

Compressors. A compressor C maps a history and a budget B to a context $c = C(h_{1:t}, B)$: a sequence of *units*, each either a (possibly truncated or masked) turn or a derived record, with total length at most B tokens (a fixed system prompt is kept and not charged).

Ideal reader and outcomes. To separate what a compressor removes from what a particular LLM can exploit, we evaluate with an *ideal reader* R^* that, for a key k, finds in c the evidence of the most recent write to k that is still present. Writes carry *evidence*: a set of normalized lines that identify the value (Section 6); a write is present if some unit contains at least a fraction $\theta = 0.8$ of its evidence lines. A *state probe* (k, t) then has one of three outcomes:

- **correct**: the latest write to k is present;
- **stale**: the latest write is absent but an older write with a different value is present—a capable reader would act on outdated state;
- **missing**: no write to k is present.

We report, for each compressor and budget, the fraction of probes in each class. Stale outcomes are strictly worse than missing ones: a missing value can be re-acquired (e.g. by re-running a command) if the agent notices the gap, whereas a stale value gives no signal that anything is wrong.

Use and copy probes. State probes require a definition of keys, which could favor a method built on the same keys. We therefore also use two key-free, behavior-grounded probes. At each decision point we take (i) the identifiers (names, dotted names, paths) and (ii) the verbatim lines (at least 12 characters) of the agent's *next action* that already occurred earlier in the history but not in the system prompt, and ask whether each is still *available* in the compressed context. We focus on *long-range* probes, whose last occurrence is more than two steps back (outside the turns every compressor keeps). These probes measure whether compression removes information the agent actually went on to use; copied lines are typically code the agent re-types from an earlier view or edit.

4 Theory

Proofs are in Appendix A. Write L_t for the *compacted log*: for every key written up to time t, the record of its latest write.

Proposition 1 (Sufficiency and size of the compacted log)**.** *For every key k, $R^*(\mathrm{L}_t, k) = S_t(k)$. Its size $|\mathrm{L}_t| = \sum_{k \in \mathcal{K}_t} |\mathrm{rec}(k)|$ depends only on the set $\mathcal{K}_t$ of keys written so far and on their current values, not on the number of turns or on the number of writes. Conversely, any compressor whose context lets R^* answer every probe correctly, for all histories over K keys with values in $\mathcal{V}$, must use contexts of at least $K \log_2 |\mathcal{V}|$ bits for some history.*

Thus the compacted log is optimal up to the encoding of keys and values, and the cost of a full-retention memory grows with the number of *live keys*, not with the horizon. Long agent runs are long mostly because the same keys are rewritten (tests re-run, files re-viewed); compaction removes exactly this redundancy.

Definition 1 (Supersession consistency)**.** A compressor is *supersession-consistent* (SC) if, for every key, the set of its writes present in the context is a suffix of its write sequence: whenever a write to k is present, every later write to k is present too.

Proposition 2 (SC compressors are never stale)**.** *If C is SC, every probe outcome is correct or missing. A recency window that keeps whole turns is SC; so is the compacted log with a perfect extractor; and so is any union of SC contexts.*

Proposition 3 (Recency windows forget by age, not importance)**.** *A recency window of W tokens answers a probe on k correctly iff the latest write to k lies within the last W tokens. If tool steps have i.i.d. lengths with mean μ and k is written as a Poisson process with rate λ_k per step, then, approximating the token distance by μ times the age in steps, the stationary probability that k is retained is $1 - e^{-\lambda_k W/\mu}$.*

Keys that are written rarely—the task statement, a constraint, the location of the bug found early on—are exactly the ones a window drops first, however important they are. Masking observations raises the effective W for keys written in agent turns (edits) but not for keys that live in tool outputs.

Proposition 4 (Stale rate of supersession-blind selection)**.** *Let a compressor keep each turn independently with probability ρ (independently of which writes it carries). For a key with m writes in distinct turns,*

$$\Pr[\textit{correct}] = \rho, \qquad \Pr[\textit{stale}] = (1-\rho)\big(1-(1-\rho)^{m-1}\big), \qquad \Pr[\textit{missing}] = (1-\rho)^m. \tag{2}$$

Hence the stale rate increases monotonically in m towards $1-\rho$, and among probes for which some *value is present, the fraction that is stale tends to $1-\rho$ as $m \to \infty$.*

Two consequences matter in practice. First, salience scores (BM25, token importance, attention) do not depend on supersession, so selection methods approach Equation (2): under tight budgets, *most* of the values they retain for frequently updated keys are outdated. Second, the same formula governs a ledger fed by an imperfect extractor: if each write is detected independently with recall r, the ledger behaves like independent selection with $\rho = r$ and stops being SC. Extraction errors therefore surface as stale state, not merely as missing state—the motivation for keeping a raw recent window next to the ledger (Section 5).

Corollary 1 (A ledger makes selection safe)**.** *Let $c = \mathrm{L}' \cup c'$, where $\mathrm{L}' \subseteq \mathrm{L}_t$ is any subset of the compacted log and c' is* any *other context (e.g. turns chosen by relevance). A probe on k can be stale only if k's record is not in L'. In particular, with the full compacted log, every probe is correct regardless of c'.*

The corollary licenses a hybrid: once the latest value of each key is pinned in the ledger, the remaining budget can be spent on supersession-blind but otherwise useful selection (e.g. relevance to the current step) without creating stale state.

Proposition 5 (Budgeted ledgers)**.** *If the compacted log exceeds the budget and key k is probed with probability p_k, maximizing the expected number of correct probes is a 0/1 knapsack with values p_k and weights $|\mathrm{rec}(k)|$. Greedy selection by $p_k/|\mathrm{rec}(k)|$ is optimal for the fractional relaxation and, combined with the best single record, a $\frac{1}{2}$-approximation (Dantzig, 1957).*

Key-level compaction removes *superseded* values. Agent histories also contain *repeated* content: the same file window is viewed several times, the same test output recurs, and agents re-type lines they wrote before. Let *content compaction* keep every distinct line only at its latest occurrence, except in turns that carry the current value of some key.

Proposition 6 (Supersession-aware content compaction is safe)**.** *Content compaction preserves the set of distinct lines in the history, so any line-level evidence (e.g. an identifier or a line the agent later copies) is still present. Because turns carrying a key's current value are left intact, for any selection of turns a state probe that is correct without content compaction is still correct with it, while its token cost shrinks by the size of the removed repeats (up to one constant-size marker per removed run).*

The saving is large in practice: 23–55% of the tokens of the agent trajectories we study are lines that occur again later (Section 7.1).

5 State-Keyed Compaction

SKC turns the analysis into a compressor with three layers of compaction (Algorithm 1): *key-level* (keep the current value of every state variable), *content-level* (drop repeated lines), and *utility-level* (spend the remaining budget on what the next step is most likely to need, without ever introducing stale state).

Extraction. An extractor maps each turn to writes. For tool-using agents most writes are *structural*: they follow from the tool call itself, which the framework already has in parsed form. For SWE-agent we derive keys deterministically from actions: `edit:<file>#<region>` for the text written by an edit (regions are tracked so that only edits to overlapping lines, or to text produced by an earlier edit, supersede each other), `view:<file>@<window>` for file content shown to the agent, and `search:<query>` and `cmd:<command>` for the outputs of searches and (normalized) shell commands. Free-text writes (e.g. a user revising a constraint) require a learned or rule-based extractor; we model its errors by its recall r.

Algorithm 1 State-Keyed Compaction (SKC)

Require: history $x_{1:t}$ (task x_1), budget B, extractor E, utility model $\hat{p}$, R, λ
$c \leftarrow [x_1] \,\|\, x_{t-R+1:t}$ ▷ pinned task and recent turns
L $\leftarrow \{\}$; **for** $i = 2..t$, **for** $(k, v) \in E(x_i)$: L$[k] \leftarrow (v, i)$ ▷ layer 1: key-level
$\tilde{x}_i \leftarrow \textsc{Dedup}(x_i \mid x_{i+1:t})$ for older turns not carrying a current value ▷ layer 2: content-level
$\mathcal{I} \leftarrow \{\textsc{Record}(k)\}_{k \in \mathrm{L}} \cup \textsc{Chunks}(\tilde{x}_{2:t-R})$
have $\leftarrow$ keys whose current value is in c
for $i \in \mathcal{I}$ in decreasing order of Equation (3) **do** ▷ layer 3: utility
  $i' \leftarrow i$ without lines already emitted for its source turn ▷ redundancy-aware
  if $i' \neq \emptyset$ **and** $|c| + |i'| \leq B$ **and** superseded keys of $i \subseteq$ *have* **then**
    $c \leftarrow c \cup \{i'\}$; *have* $\leftarrow$ *have* $\cup$ current keys of i
  end if
end for
return c in original order (ledger records first, chunks of a turn merged)

Layer 1: key-level compaction (ledger). For every key, SKC keeps only the latest extracted write as a *ledger record*, rendered *verbatim* (values capped at V=300 tokens, keeping head and tail) with provenance (`[state] key (as of turn i)`). Verbatim values avoid the paraphrase drift of LLM summaries; provenance lets a reader order records.

Layer 2: content-level compaction. Older turns are deduplicated as in Proposition 6: a line is dropped if it occurs again later (or in the task or the recent window), and a dropped run is replaced by a marker (`[...]`); turns that carry the current value of a key are exempt.

Layer 3: utility-guided, supersession-safe selection. The task and the last R=2 turns are kept verbatim. Everything else becomes an *item*: a ledger record, or a chunk of at most 12 lines (200 tokens) of a deduplicated older turn. A small model predicts, for every item, the probability $\hat{p}$ that the next step needs it (contains an identifier or a line the next action uses), from 31 model-agnostic features computed at decision time: age, size, item type (agent/tool chunk by tool kind, ledger record by key kind), whether the item carries a current or a superseded value, BM25 relevance to the task and the latest agent turn (jointly and to the latest turn alone), identifier and line overlap with the latest agent turn, observation and task, the rarity of shared identifiers, and how often and how recently its identifiers were referred to by later agent turns. Items are admitted greedily by

$$\big(\hat{p}_i + \lambda \mathbb{1}[i \text{ is a ledger record}]\big) \,/\, |i|, \tag{3}$$

the knapsack ratio of Proposition 5, where λ=0.3 is the value of holding the current state even when the very next step does not use it. Selection obeys the *supersession-safety constraint* of Corollary 1: an item that carries a superseded value of key k is admitted only if the current value of k is already in the context. Selection is also *redundancy-aware*: a ledger record and the chunks of the same turn often repeat each other (a record is the verbatim value carried by that turn), so each item is charged only for the lines not yet emitted for its source turn, those lines are replaced by a marker, and an item with no new line is skipped. On the development set this removes duplicates that took 6–9% of the context. Selected chunks of the same turn are merged in their original order, with gaps marked.

Training the utility model. Labels come for free from logged trajectories: an item is positive if the agent's next action uses one of its identifiers or copies one of its lines. We use gradient-boosted trees (scikit-learn's histogram-based gradient boosting) and train them *across models*: the model applied to a trajectory of LLM m on SWE-bench instance x is trained only on the other four LLMs and on the other half of the instances. The utility model thus never sees the evaluated LLM or task.

SKC calls no LLM: extraction is a parse of the tool call, compaction is a dictionary update and a set lookup, and the utility model is an ensemble of small trees over 31 features. It runs in 0.12–0.31 s per step on one CPU core (Table 7), and since it always selects from the original history (rather than from a previous summary) it does not compound errors over time, unlike recursive summarization. We also report **SKC-basic**, an

earlier heuristic variant without layers 2–3 (ledger with a type/recency priority, a raw window and an action skeleton; Appendix C).

6 Experimental Setup

Trajectories. We use public trajectories submitted to the SWE-bench leaderboards (Jimenez et al., 2024) (Table 1). *In-domain*: SWE-agent (Yang et al., 2024) on SWE-bench Lite with five LLMs—GPT-4, Claude 3 Opus and GPT-4o (SWE-agent 0.x interface), Claude 3.7 Sonnet and Claude 4 Sonnet (1.x interface)—1,479 trajectories. *Cross-scaffold*: SWE-bench Verified runs of OpenHands (Wang et al., 2025) with Claude 4 Sonnet, GPT-5, Kimi K2, Qwen3-Coder-480B and Devstral-Small, and of SWE-agent with SWE-agent-LM-32B and Kimi K2—3,500 trajectories, including three open-weight models and two log formats (native and prompt-based function calls). A further OpenHands export (Claude Opus 4.5) records every tool call as "`Action: unknown`" and is excluded because no actions can be recovered. All token counts use a real BPE tokenizer (the legacy Claude tokenizer distributed with the MIT-licensed `anthropic` SDK).

Splits and protocol. The first ten instances (alphabetically) of every Lite model form a *development* set used for all design and hyper-parameter decisions; results are reported on the remaining 1,429 Lite trajectories. For the cross-scaffold set we sample 150 trajectories per configuration (1,050) and drop instances that also occur in SWE-bench Lite. SKC's utility model is trained with leave-one-LLM-out, instance-disjoint folds on Lite (Section 5); on the cross-scaffold set it is a single model trained on all Lite trajectories, so neither the scaffold, the LLM nor the task was seen in training. For each trajectory and budget $B \in \{4\text{k}, 8\text{k}, 16\text{k}, 32\text{k}\}$ we evaluate up to six decision points whose history exceeds B, compressing the history with every method and probing state (all keys written so far), use and copy (Section 3). We report per-trajectory macro averages with 95% bootstrap confidence intervals and paired bootstrap tests over trajectories.

Baselines. Seven model-free compressors covering the families used in practice (settings in Appendix C): **recency window** (FIFO truncation, task pinned); **observation masking**, which keeps all agent turns and masks all but the last ten tool outputs (Lindenbauer et al., 2025); **truncate observations**, which caps older tool outputs; **BM25** over whole turns and over the same 12-line **chunks** SKC uses (salience-based selection (Robertson & Zaragoza, 2009)); **self-information** line pruning, a model-free variant of Selective Context (Li et al., 2023); and **random** selection. **SKC-basic** is the heuristic variant of SKC without layers 2–3. LLM summarization is evaluated in the LLM-in-the-loop study.

LLM-in-the-loop study. To check that what the ideal reader finds is also usable by a model, we sample 60 decision points per Lite LLM (300 in total, 8k budget) and give the compressed context to three reader LLMs from two families (Qwen3.8-Flash, Qwen3.7-Flash, DeepSeek-V4-Flash; thinking disabled, temperature 0). Task A (*state QA*, 217 points) asks for the last line of the most recent output of a command last run at least three steps earlier; answers are graded correct, stale (an earlier run's output), unknown or wrong. Task B (*next action*) asks for the agent's next tool call; we report whether it names the same file as the true action and identifier-level F1. An **LLM summary** baseline replaces the history before the last two turns by a summary of at most 4k tokens written by the same reader model. This study (and the ALFWorld study in Appendix E) was run with SKC before redundancy-aware selection was added, a variant that keeps 2–4 points less state in the ideal-reader evaluation.

7 Results

7.1 What long agent contexts are made of

Table 1 and Figure 1 characterize the 4,979 trajectories. Tool outputs make up 65–89% of all tokens. Between 23% and 55% of tokens are lines that occur again later in the same history (repeated file views, re-run tests, re-typed code), redundancy that is invisible to per-turn compressors but removed by content compaction (Proposition 6). Recent LLMs run longer and reach further back: for OpenHands with Qwen3-Coder or Claude 4 Sonnet the median trajectory has 64–66 steps and 41–42k tokens, and 80–81% of the identifiers their

Table 1: Trajectories used in this study. Tokens are BPE tokens of the full trajectory (task, agent and tool turns). *Tool*: share of tokens in tool outputs. *Rep.*: share of tokens in lines that recur later in the same trajectory. *Far*: share of identifiers used in an action that first appeared more than 10 steps earlier. SA: SWE-agent; OH: OpenHands.

Benchmark	LLM (scaffold)	Traj.	Steps med.	Steps p90	Tokens median	Tool %	Rep. %	Far %
Lite	GPT-4 (SA 0.x)	300	20	34	15.4k	79	40	33
	Claude 3 Opus (SA 0.x)	300	18	23	8.8k	82	39	21
	GPT-4o (SA 0.x)	278	40	65	25.7k	88	55	67
	Claude 3.7 Sonnet (SA 1.x)	300	29	54	37.2k	84	23	61
	Claude 4 Sonnet (SA 1.x)	301	59	91	41.7k	73	31	79
Verified	Claude 4 Sonnet (OH)	500	64	106	41.3k	71	35	80
	GPT-5 (OH)	500	28	47	28.6k	89	25	63
	Kimi K2 (OH)	500	52	84	37.8k	70	35	76
	Qwen3-Coder-480B (OH)	500	66	116	42.5k	65	35	81
	Devstral-Small (OH)	500	42	97	33.2k	76	34	76
	SWE-agent-LM-32B (SA 1.x)	500	33	76	37.4k	83	29	71
	Kimi K2 (SA 1.x)	500	22	81	16.5k	84	25	66

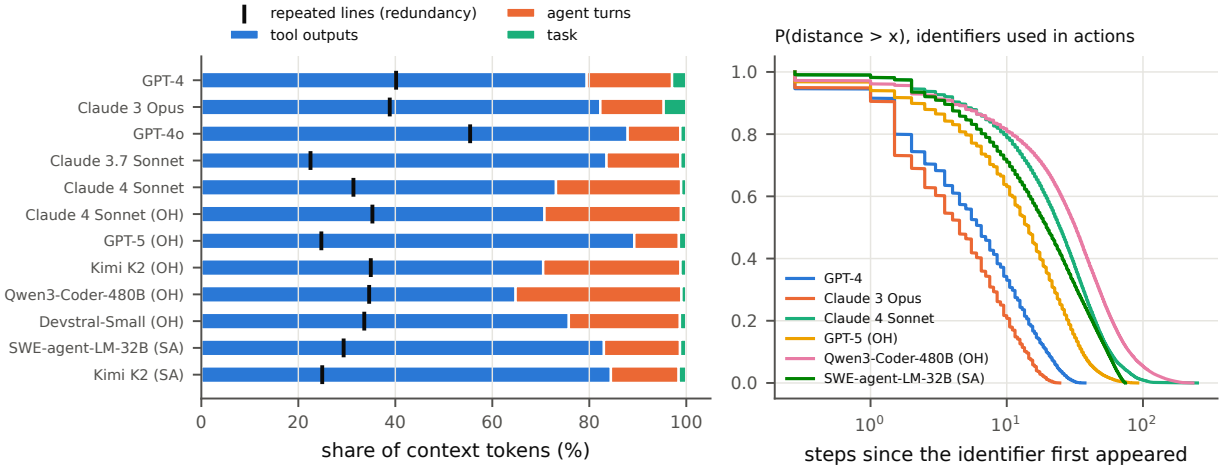


Figure 1: Left: token composition per configuration; ticks mark the share of tokens in lines that recur later (redundancy). Right: survival function of the distance between an identifier's first occurrence and its use in an action.

actions use first appeared more than ten steps earlier (21% for Claude 3 Opus). A median trajectory rewrites 3–7 state keys at least once. Long horizons therefore combine long-range dependencies, heavy repetition and frequent overwrites—the three conditions our analysis identifies as hard for windows (forgetting by age), for per-turn selection (staleness) and for budget efficiency (redundancy).

7.2 Main results

State retention. Table 2 and Figure 2 give the main comparison. In-domain, SKC keeps the current value of 67.1% / 78.9% / 89.1% / 94.8% of all state variables at 4k / 8k / 16k / 32k tokens, against 57.8% / 67.3% / 75.7% / 82.5% for the best baseline (random selection) and 45–72% for the recency window. The paired differences, +9.3, +11.6, +13.4 and +12.4 points, have 95% intervals of width below 1.5 points ($p < 10^{-3}$). SKC-basic, which spends its budget on the ledger first, keeps more state only at the smallest budget (69.7% at 4k); from 8k on SKC keeps more (+1.3 to +7.0 points, $p < 10^{-3}$). The gain transfers to unseen scaffolds and LLMs: on the cross-scaffold set SKC improves on the best baseline by +6.7, +9.3, +10.1 and +10.2 points (all $p < 10^{-3}$), for each of the seven configurations (Appendix D); there SKC-basic keeps more at 4k

Table 2: Ideal-reader evaluation on held-out trajectories (per-trajectory macro averages, %). Columns are budgets of 4k / 8k / 16k / 32k tokens. Best per column in bold. 95% bootstrap half-widths are below 0.8 points for state correct, 0.2 for stale, and 2–4 points for use and copy (full intervals in the released tables).

	State correct ↑				State stale ↓				Use, long range ↑				Copy, long range ↑			
(a) In-domain: SWE-agent on SWE-bench Lite, 5 LLMs, 1,429 test trajectories																
Recency window	45.3	55.5	64.4	72.4	0.3	0.2	0.2	**0.1**	54.5	76.1	91.4	95.0	48.6	75.4	93.0	96.1
Observation masking	44.8	50.8	46.4	39.1	0.3	0.3	0.6	0.7	55.0	77.7	87.5	88.3	49.5	78.4	93.2	93.5
Truncate observations	49.2	54.8	58.8	59.9	0.7	1.3	1.8	1.8	70.1	84.7	89.1	88.2	66.3	87.0	94.1	95.7
BM25 (turns)	47.7	53.6	60.6	70.7	1.9	1.8	1.8	1.3	72.7	80.4	87.3	92.3	64.0	74.0	86.0	93.4
BM25 (chunks)	48.5	56.6	65.1	75.1	2.1	2.0	1.7	1.3	80.5	89.4	95.4	**99.0**	**75.7**	**87.7**	**95.3**	**99.0**
Self-information	36.8	44.7	54.4	69.3	1.3	1.6	1.8	1.4	58.3	73.6	87.0	95.8	34.4	48.2	69.0	85.4
Random selection	57.8	67.3	75.7	82.5	2.8	2.2	1.6	1.0	74.0	82.2	91.6	95.4	63.4	77.0	90.0	96.7
SKC-basic (ours)	**69.7**	77.6	83.6	87.8	**0.2**	**0.2**	0.2	0.1	72.8	83.7	92.7	94.9	51.3	73.4	90.6	98.2
SKC (ours)	67.1	**78.9**	**89.1**	**94.8**	**0.2**	**0.2**	**0.1**	**0.1**	**81.5**	**90.3**	**95.9**	**99.0**	70.5	84.6	94.2	97.2
(b) Cross-scaffold: OpenHands and SWE-agent on SWE-bench Verified, 7 LLM configurations, 1,050 trajectories																
Recency window	37.5	51.3	66.3	78.7	**0.2**	0.2	0.2	**0.1**	51.7	70.0	85.2	94.7	48.9	71.7	88.2	96.5
Observation masking	37.0	46.3	47.2	42.2	**0.2**	0.2	0.3	0.5	52.1	72.7	86.9	91.9	49.6	75.1	89.5	94.7
Truncate observations	36.8	46.1	53.0	56.0	0.5	0.8	1.2	1.3	59.9	76.2	86.5	92.7	57.8	**78.8**	88.9	95.9
BM25 (turns)	34.0	43.4	58.0	75.4	1.0	1.3	1.3	0.8	64.7	77.7	86.1	93.1	51.5	65.4	74.3	83.2
BM25 (chunks)	29.4	42.4	59.9	77.2	0.7	1.1	1.2	1.0	**69.0**	**83.8**	**93.7**	98.1	**59.0**	76.8	90.5	97.3
Self-information	25.1	31.2	44.1	64.1	0.3	0.6	0.9	1.5	43.2	62.2	80.3	92.4	20.7	30.0	46.4	66.3
Random selection	43.8	56.8	71.9	84.5	1.4	1.5	1.3	0.9	63.0	76.0	87.2	94.7	50.6	65.1	81.3	91.7
SKC-basic (ours)	**55.4**	**70.2**	79.5	85.2	**0.2**	**0.1**	**0.1**	**0.1**	61.4	74.1	86.2	93.7	46.2	63.6	83.8	94.3
SKC (ours)	50.5	66.2	**82.0**	**94.8**	**0.2**	0.2	**0.2**	**0.1**	66.7	82.2	93.4	**98.9**	57.2	78.2	**92.9**	**98.2**

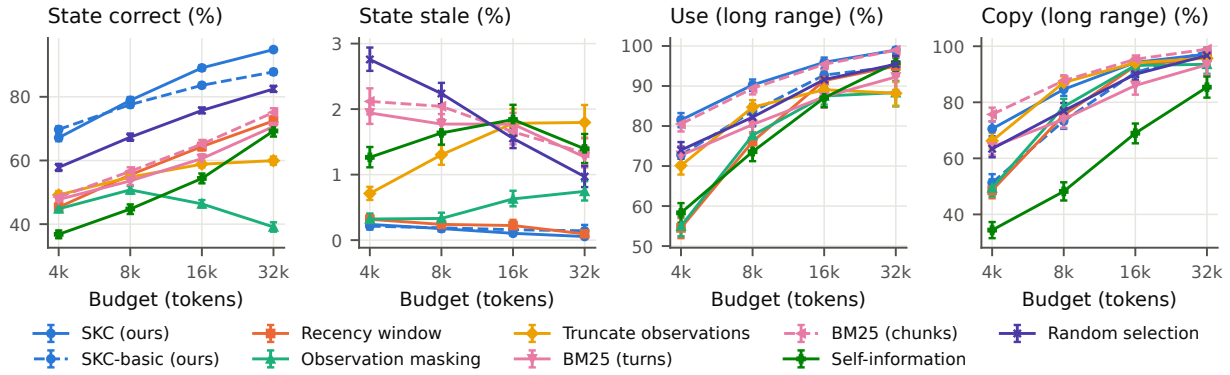


Figure 2: In-domain results as a function of the budget (95% bootstrap intervals). SKC and SKC-basic keep the most state at every budget and are as stale-free as a recency window; SKC matches fine-grained BM25 on identifiers the next action uses, and BM25 keeps the most copied lines.

and 8k. Budgets correspond to real compression: at 4k, SKC reduces the average history of 18.6k tokens by 4.7× (at 32k, 1.7×) in 0.12–0.31 s per step on one CPU core (Table 7).

Redundancy-aware selection. Charging each item only for the lines not yet emitted for its turn (Section 5) improves every probe at every budget on both sets: in-domain by +2.1 to +4.1 points of state, +1.4 to +2.0 of use and +0.9 to +4.2 of copy; on the cross-scaffold set by +1.7 to +6.8, +1.8 to +3.1 and +1.6 to +4.1 (paired tests against SKC without it, all $p < 0.003$). It costs no LLM call and about 25% more selection time.

Staleness. Selection methods leave stale state on 1.0–2.8% of all probes in-domain. This average hides the dependence on overwrites predicted by Proposition 4 (Figure 3): for keys written three or more times, BM25 and random selection are stale on 9–13% of probes, i.e. 13–18% of the values they retain for such keys are

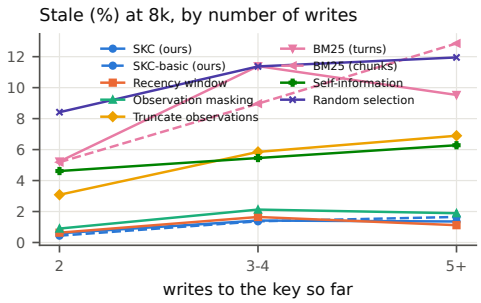


Figure 3: Stale rate at 8k (in-domain) by the number of writes the key has received. Selection methods (BM25, random) approach the regime of Proposition 4; supersession-consistent methods (window, masking) and SKC stay below 2%. Keys written once cannot be stale.

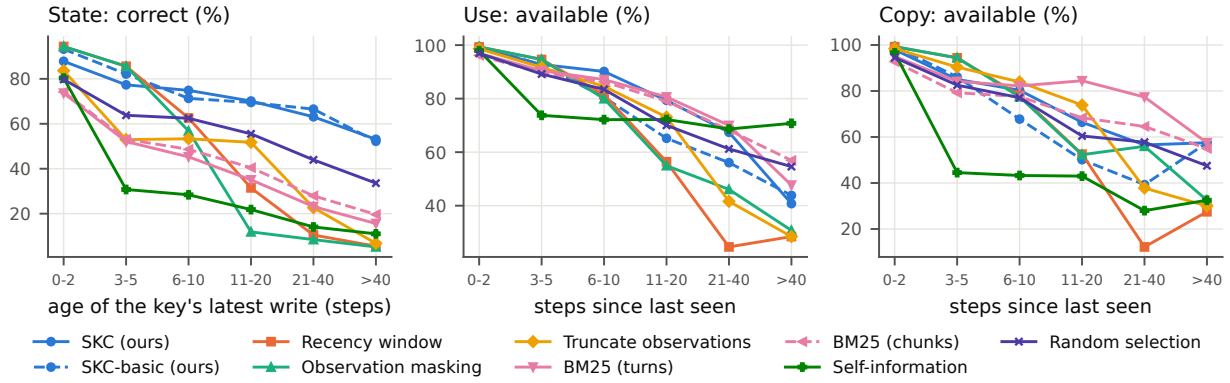


Figure 4: In-domain results at 8k by age: state probes by the age of the key's latest write (left), use and copy probes by the number of steps since the identifier or line last appeared (middle, right).

outdated. SKC is stale on 0.1–0.2% of probes, never more than the recency window (the least stale baseline; lower at 4k and 16k, $p < 0.05$, indistinguishable otherwise) while keeping 22–25 points more state.

Forgetting by age. Figure 4 breaks results down by age. Windows and masking are as good as SKC for keys written in the last five steps and collapse beyond ten, as Proposition 3 predicts; SKC degrades gracefully and still keeps most keys written 20–40 steps earlier.

Information the agent uses next. The key-free probes paint a more nuanced picture. In-domain, SKC preserves the most long-range identifiers at every budget (81.5% vs. 80.5% for chunk-level BM25 at 4k; differences of 0–1 point, $p \geq 0.2$) and far more than the window (54.5%), masking (55.0%) or random selection (74.0%). For lines the agent later *copies*, chunk-level BM25 is best in-domain (75.7% vs. 70.5% at 4k; 1.1–3.1 points at 8k–32k). On the cross-scaffold set SKC trails BM25 by 1.7–2.3 points on use at 4k–8k and matches or exceeds it from 16k, and on copy it is within 2 points at 4k–8k and better by 2.5 points at 16k ($p = 0.006$). Copying is dominated by an agent re-typing parts of a script or edit it wrote a few steps before, which lexical overlap with the latest agent turn captures directly. Figure 5 shows that this is a trade-off governed by λ rather than a ceiling. On the ablation subset at 4k, $\lambda = 0.1$ comes within 0.5 points of chunk-level BM25 on use and 1.2 on copy while keeping 12.7 points more state (61.4% vs. 48.7%), and $\lambda = 1$ dominates SKC-basic, with more state (70.6% vs. 69.7%), use (+7.6) and copy (+11.0). The oracle utility in Section 7.4 shows that better utility prediction alone would raise use and copy to 93–95%.

7.3 Can an LLM use what is kept?

Table 3 and Figure 6 show that the ideal-reader gains are usable by real models. With SKC's context, readers answer 77.0% / 50.2% / 59.9% of the state questions correctly, against 40.1% / 24.9% / 27.2% with the best baseline; the improvements (+25 to +37 points over the window, +29 to +45 over masking and BM25) are significant for all three readers ($p < 10^{-3}$) and come mainly from turning "unknown" answers into correct

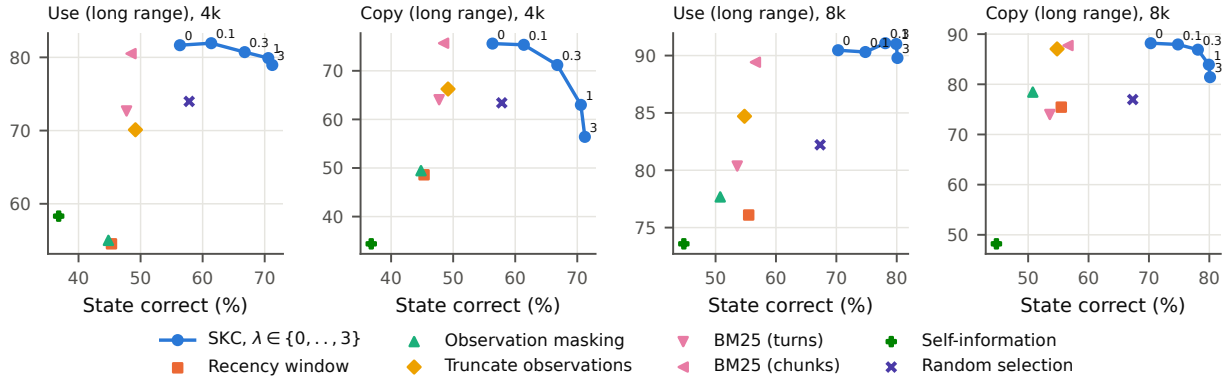


Figure 5: Trade-off between state retention and use/copy probes as the ledger bonus λ varies (in-domain, 100 trajectories per LLM), with baselines from the main evaluation. Numbers label λ.

Table 3: LLM-in-the-loop evaluation at 8k (300 decision points, 217 state questions). QA: % correct; F1: next-action identifier F1 (%). "Summ." is an LLM summary of the older history plus the last two turns (≈2.4k tokens); "Summ.+recent" fills the rest of the budget with the most recent turns (≈7.5k tokens), as production agents do. *: SKC differs from the method with $p < 0.01$ (paired bootstrap).

	Window		Obs. mask.		BM25		Summ.		Summ.+recent		SKC (ours)	
Reader	QA	F1	QA	F1	QA	F1	QA	F1	QA	F1	QA	F1
Qwen3.8-Flash	40.1*	47.9	31.8*	47.9	31.8*	46.1	2.3*	29.9*	37.8*	45.1	**77.0**	45.7
Qwen3.7-Flash	24.9*	34.2	19.4*	**37.0***	20.7*	30.2	4.1*	22.2*	18.0*	34.6	**50.2**	31.6
DeepSeek-V4-Flash	27.2*	**40.5**	24.9*	39.7	24.4*	38.8	6.5*	27.8*	25.3*	40.9	**59.9**	37.2
QA stale (%)	0.5–1.4		0.0–1.4		3.2–4.1		0.0		0.5–1.4		0.9–2.3	

ones. LLM summaries, although written by the same model, answer only 2–7% of the questions: they describe what was done ("ran the reproduction script, which still failed") rather than preserving the exact state, the paraphrase drift that verbatim ledger values avoid. Part of this gap is budget: the summaries use only 0.5–0.9k tokens. The budget-matched variant that fills the remaining budget with the most recent turns (*Summ.+recent*, ≈7.5k tokens, the design of production agents) recovers 18–38%—but still trails the plain recency window by 2–7 points and SKC by 32–39 points ($p < 10^{-3}$ for all readers): spending part of the budget on a summary loses more exact state than it preserves. For next-action prediction, SKC is statistically indistinguishable from window, masking, BM25 and Summ.+recent in 23 of 24 comparisons (exceptions: 5.3 F1 points below masking for Qwen3.7-Flash, and 10.3 points of file-path hits below Summ.+recent for DeepSeek-V4-Flash, $p = 0.03$) and better than the short LLM summaries by 9–16 F1 points ($p < 10^{-3}$). SKC thus makes the agent's current state readable without degrading what the next step needs.

7.4 Ablations

Table 4 isolates each component. *Redundancy-aware selection* adds 2.1 / 4.0 points of state, 1.9 / 2.4 of use and 3.3 / 3.1 of copy at 4k / 8k. Starting from SKC without it, the *ledger* is what retains state: removing it costs 10.3 / 6.7 points at 4k / 8k, while its budget would otherwise buy only 0.3–1.5 points of use and 2.5–7.3 of copy. *Deduplication* adds 2.0 / 4.7 points of state at little cost elsewhere, consistent with Proposition 6. The *learned utility* buys 8.7 points of copy at 4k over the heuristic priority, at the price of 4.8 points of state—the same trade-off that λ controls. The *safety constraint* lowers staleness (0.5% → 0.3% at 4k) at no measurable cost. An imperfect extractor (recall 0.7) costs 5 points of state and triples staleness, the effect predicted by Proposition 4. Finally, an *oracle* utility that knows what the next action uses lifts use and copy to 93–95% with the same budget, framework and state retention: the remaining gap to chunk-level BM25 on copy is a matter of utility prediction (AUC 0.818 for our model vs. 0.707 for BM25 relevance, Table 8), not

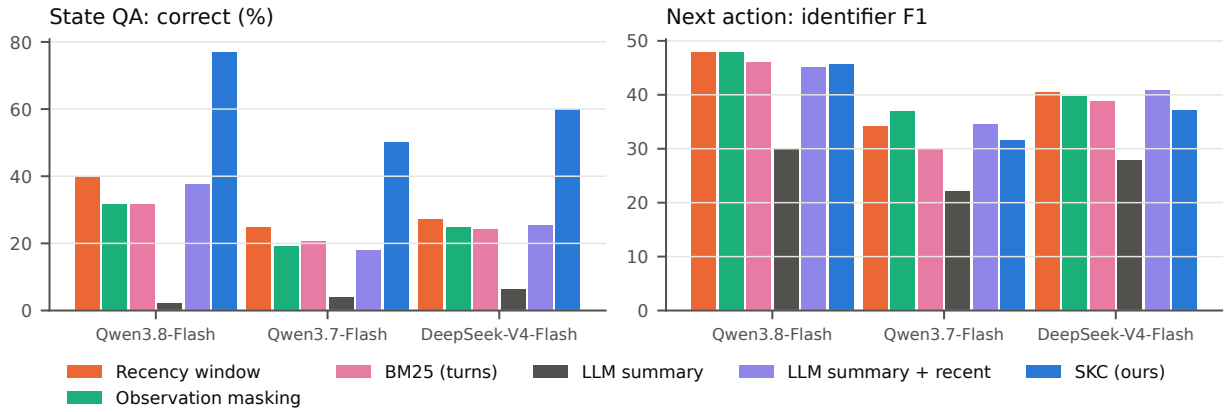


Figure 6: LLM-in-the-loop evaluation: state-question accuracy (left) and next-action identifier F1 (right) for three reader LLMs.

Table 4: Ablations (in-domain, 100 trajectories per LLM, %). The component ablations below the first block remove one component from SKC *without* redundancy-aware selection (the configuration they were run on); the oracle uses the same configuration.

	State		Stale		Use		Copy	
Variant	4k	8k	4k	8k	4k	8k	4k	8k
SKC	66.8	78.1	0.2	0.2	80.7	91.1	71.2	86.9
$\lambda = 0$ / $\lambda = 3$	56.3/71.2	70.3/80.1	0.2/0.3	0.2/0.2	81.7/79.0	90.5/89.8	75.6/56.4	88.2/81.4
− redundancy-aware selection	64.7	74.1	0.3	0.2	78.8	88.7	67.9	83.8
− ledger (layer 1)	54.4	67.4	0.3	0.2	80.3	89.0	75.2	86.3
− deduplication (layer 2)	62.7	69.4	0.4	0.5	78.1	88.4	69.0	83.3
− learned utility (heuristic)	69.5	76.6	0.3	0.3	78.1	88.3	59.2	82.9
− chunking (whole turns)	67.0	77.1	0.3	0.2	78.6	87.8	65.1	79.8
− safety constraint	64.4	73.7	0.5	0.3	79.2	88.9	67.8	84.4
logistic-regression utility	65.5	75.0	0.2	0.2	80.0	88.5	71.2	82.3
extractor recall 0.7	59.7	69.0	0.9	0.9	79.1	88.9	69.5	84.3
Oracle utility (upper bound)	66.2	75.9	0.3	0.2	93.3	95.3	93.1	94.5

of the compaction layers. Adding features that describe where the agent is working (the file and region of the latest view or edit, BM25 against the latest observation) raised the AUC only to 0.824 and did not improve the development-set probes, so we did not adopt them.

7.5 Controlled experiments

StateTrack lets us vary one factor at a time and compare with the theory (Figures 7 to 9; details in Appendix F). Pinned, slowly changing and frequently changing keys behave as Proposition 3 predicts: windows keep recently written keys and lose slow ones; selection keeps salient keys but is stale on 13–30% of probes; the ledger keeps 90% of all keys at 2k tokens and 100% from 4k. Pinning the task is itself not supersession-consistent—a user's revision of a pinned constraint is lost while the original stays pinned—which explains the 6% stale rate of windows. As trajectories grow from 50 to 800 steps at 4k, window accuracy falls from 48% to 28% and BM25 staleness rises from 24% to 43%, while the ledger stays at 98–100% (Proposition 1). The stale rate of random selection follows Equation (2) at the measured keep rate (Figure 8), and so does the stale rate of a ledger fed by an extractor of recall r (Figure 9); a raw recent window hedges against missed writes (at $r = 0.7$, 9% stale instead of 25%), whereas a relevance-based extension does not (16%), as Corollary 1 implies.

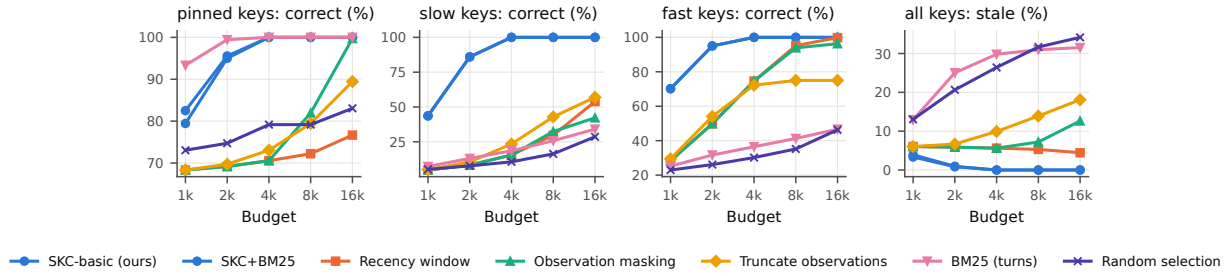


Figure 7: StateTrack (30 seeds): % correct per key family and % stale over all keys vs. budget. The ledger-based SKC-basic and its BM25 hybrid overlap.

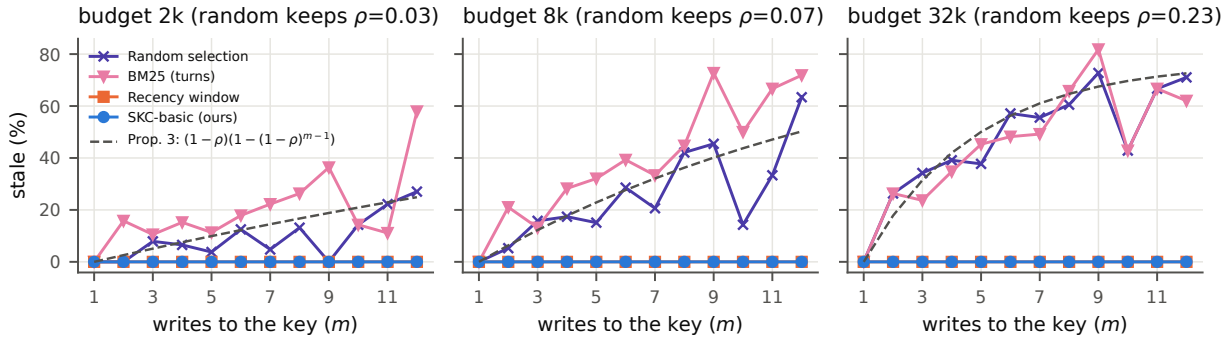


Figure 8: Stale rate vs. number of writes per key in StateTrack. Dashed: Proposition 4 at the measured keep rate ρ.

8 Discussion and Limitations

When to use which variant. SKC is the default: from 8k tokens on it keeps the most state in-domain and the most identifiers the next step uses. SKC-basic keeps more state at the smallest budgets and on unfamiliar scaffolds (4k–8k cross-scaffold), where the learned utility spends budget on content the next step may need; λ moves continuously between the two, and $\lambda = 1$ already dominates SKC-basic in-domain. For agents whose next action often re-types recent text, a lexical relevance term remains a strong signal—our utility model already uses it, and the oracle bound shows how much better utility prediction could still gain.

From sufficiency to success. The ideal reader measures whether information is present; the LLM-in-the-loop study shows that three different readers do use it. End-to-end on ALFWorld (Appendix E), SKC does not differ significantly from a window, observation masking or an LLM summary at any of four budgets (300–1200 tokens), even though compression itself costs 13–18 points of success at 300–400 tokens; the games are short (mean prompt ≈400 tokens) and failures there come mostly from planning rather than missing state. We did not run SWE-bench agents end-to-end on compressed contexts: that requires containerized environments and many agent runs per method, and is the most important next step. Our analysis predicts that compressors with stale or missing state cause agents to repeat work or act on outdated results; the ALFWorld experiment neither confirms nor refutes this.

Probe design. State keys are derived by the same structural parser that SKC uses, which favors SKC on state probes; we therefore also report key-free use and copy probes (on which SKC is not uniformly best), the LLM-in-the-loop study, exact ground truth on StateTrack, and noisy extractors. The ordering of methods is unchanged for evidence thresholds $\theta \in \{0.6, 0.8, 1.0\}$ (Appendix D). The three v2 utility features (BM25 against the latest agent turn alone and line overlap with recent turns) were added after a first evaluation and validated on the development split only; on the test set they change SKC by less than

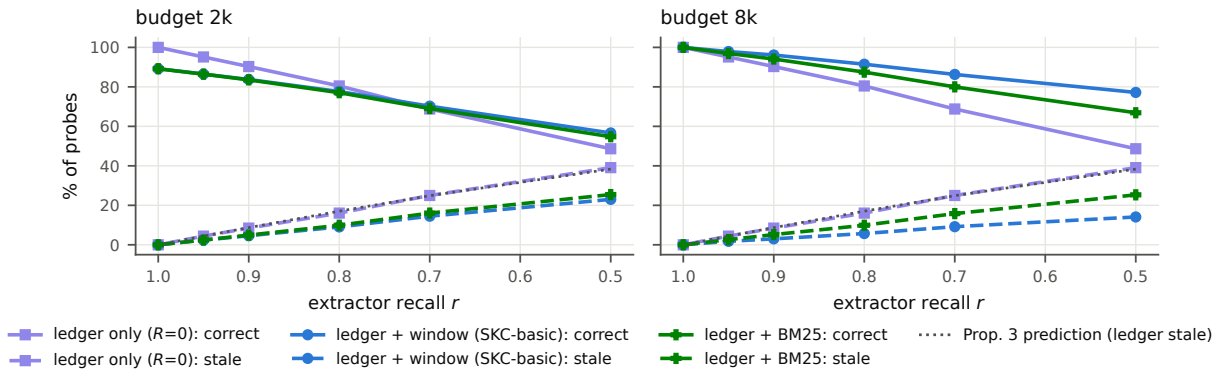


Figure 9: Ledger variants under extractor recall r (StateTrack). Solid: correct; dashed: stale; dotted: Proposition 4 with $\rho = r$ for the ledger alone.

one point. Redundancy-aware selection was likewise designed on the development split, after a diagnosis of where SKC's budget went (20–25% of its context tokens were duplicate lines), and then evaluated once on the test sets.

Scope. The trajectories come from software-engineering agents with typed tools, where most state writes can be extracted structurally. Agents whose state lives in free text (dialogue, browsing) need learned extractors, whose recall then governs staleness (Proposition 4). LLM summaries were evaluated only in the LLM-in-the-loop study, with one summary written per decision point.

9 Conclusion

Agent histories are event logs, and what an agent must keep is the current state those events define. Viewing compression through this lens separates missing from stale state, explains when common compressors fail—windows forget by age, supersession-blind selection retains outdated values at a rate that grows with overwrites—and suggests compaction at three levels: keys, content and utility. SKC, a model-free implementation, keeps 9–13 points more state than the best of seven baselines in-domain and 7–10 points more on unseen scaffolds and LLMs, is as stale-free as a recency window, preserves as many of the identifiers the next action uses as the best lexical selector in-domain, and doubles the accuracy with which LLM readers recover the agent's current state, without hurting next-action prediction. The open problems are utility prediction, which an oracle shows could close the remaining gap on copied content, extraction for unstructured state, and end-to-end validation.

Broader Impact Statement

This work makes long-running LLM agents cheaper and more reliable by keeping their working state accurate under context limits. Stale or missing state can lead agents to act on outdated information—for example to report a test as passing after a later run failed—so methods that surface the current state, and evaluations that measure staleness, can reduce such silent errors. More capable autonomous agents also amplify the consequences of misuse and of unattended mistakes; our method does not change what an agent is permitted to do, and we recommend that compressed contexts keep provenance (as SKC's ledger does) so that humans can audit what an agent knew when it acted. All data used are public agent trajectories released by the SWE-bench leaderboard participants; no human subjects were involved.

Use of AI Assistants

An AI coding assistant was used to implement the experimental code, run the experiments and draft the manuscript under the direction of the authors, who take full responsibility for the content.

References

Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. *Journal of Computer and System Sciences*, 58(1):137–147, 1999.

Anthropic. Effective context engineering for AI agents. Anthropic Engineering Blog, 2025.

Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–288, 1957.

Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma Technical Report, 2025.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.

Najoung Kim and Sebastian Schuster. Entity tracking in language models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. In *International Conference on Machine Learning (ICML)*, 2024.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL*, 2024.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Qingyue Wang, Liang Ding, Yanan Cao, Zhiliang Tian, Shi Wang, Dacheng Tao, and Li Guo. Recursively summarizing enables long-term dialogue memory in large language models. *arXiv preprint arXiv:2308.15022*, 2023.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *AAAI Conference on Artificial Intelligence*, 2024.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.

A Proofs

Proposition 1. By construction L_t contains, for every written key, the record of its latest write and no other record for that key, so R^* returns the value of that write, which is $S_t(k)$ by definition. Each key contributes exactly one record whose size depends only on the key name and its current value, so $|\mathrm{L}_t|$ is a function of $(\mathcal{K}_t, S_t)$ alone. For the lower bound, fix K keys and consider the $|\mathcal{V}|^K$ histories that write each key once with an arbitrary value. If two such histories with different state vectors were mapped to the same context, R^* would give the same answer on both for every key and hence be wrong on at least one probe of one of them. The compressor is therefore injective on these histories, so some context has description length at least $\log_2 |\mathcal{V}|^K = K \log_2 |\mathcal{V}|$ bits. □

Proposition 2. Suppose an SC context has some write to k present. The present writes of k form a non-empty suffix of its write sequence, which contains the latest write, so R^* returns the latest value (correct). Otherwise no write is present (missing). A window of whole turns keeps a suffix of turns, hence a suffix of every key's writes. The compacted log keeps exactly the last write of each key (a suffix of length one). If c_1, c_2 are SC, the present writes of k in $c_1 \cup c_2$ are a union of two suffixes, which is a suffix. □

Proposition 3. The window contains exactly the turns whose starting position lies within the last W tokens, so the latest write to k is present iff it lies in that range; older writes are then absent too (no stale outcome), and if the latest write is outside the range so are all older ones (missing). For a stationary Poisson process with rate λ_k per step, the time since the last event (the age A) is exponentially distributed with rate λ_k. Approximating the token distance by μA, $\Pr[\mu A \le W] = 1 - e^{-\lambda_k W/\mu}$. □

Proposition 4. Let the writes of k be in turns $i_1 < \cdots < i_m$, each kept independently with probability ρ. The probe is correct iff turn i_m is kept (ρ). It is missing iff none is kept ($(1-\rho)^m$). It is stale iff i_m is dropped and at least one of the $m-1$ earlier turns is kept: $(1-\rho)(1-(1-\rho)^{m-1})$ (we assume distinct values; equal values only turn stale outcomes into correct ones). This is increasing in m with limit $1-\rho$. Conditioned on some write being present, the stale fraction is $(1-\rho)(1-(1-\rho)^{m-1})/(1-(1-\rho)^m) \to 1-\rho$. For an extractor with independent recall r feeding a latest-wins ledger, the ledger record of k is the latest *detected* write; it equals the true latest write with probability r, is an older write with probability $(1-r)(1-(1-r)^{m-1})$, and is absent otherwise, which is Equation (2) with $\rho = r$. □

Corollary 1. If k's record is in L', the latest write of k is present, so R^* answers correctly whatever else c' contains, since every other present write of k is older. □

Proposition 6. A line is removed from a turn only if the same (normalized) line occurs in a later turn, the task, or the recent window, and the latest occurrence of every line is never removed (processing turns from newest to oldest, a line is removed only once it has been seen), so the set of distinct lines of the history is unchanged. Turns carrying the current value of some key are not modified, so the evidence group of every current value is identical with and without content compaction, and a probe whose current value is present under a selection remains present under the same selection. Removed runs are replaced by one marker each. □

Proposition 5. The expected number of correct probes is $\sum_k p_k \mathbb{1}[k \text{ kept}]$ subject to $\sum_k |\mathrm{rec}(k)| \mathbb{1}[k \text{ kept}] \le B$, a 0/1 knapsack. The ratio-greedy solution of the fractional relaxation is optimal (Dantzig, 1957); the better of the greedy integral prefix and the single most valuable item achieves at least half of the fractional optimum. □

B Probe and Evidence Details

Evidence lines. Values are split into lines; each line is normalized by removing carriage returns and editor line-number prefixes (e.g. `123:` or `123\t`) and collapsing whitespace; lines shorter than 6 characters or without alphanumerics are discarded, and duplicates are removed. For command outputs the evidence is the

last 12 such lines (results, tracebacks and test summaries are printed last); for all other keys it is 12 lines sampled evenly across the value. A write is present if one unit contains at least 80% of its evidence lines. Matching is line-exact after normalization, so an edit is also present if the file view echoed after the edit shows the new lines.

Key derivation for SWE-agent. SWE-agent 0.x actions (`open`, `goto`, `scroll_up`/`scroll_down`, `create`, `edit a:b`, `search_dir`, `search_file`, `find_file`) and SWE-agent 1.x actions (`str_replace_editor` with `view`, `create`, `str_replace` or `insert`; bash) are parsed deterministically. The currently open file in 0.x is tracked from the `[File: ...]` header of observations. *Edit regions*: a line-range edit supersedes an earlier edit of the same file if their line ranges overlap; a `str_replace` supersedes an earlier edit if its `old_str` is contained in, or shares an informative line with, the text written by that edit; `create` resets the file. *View windows* are bucketed by starting line into 50-line blocks. Commands are normalized by stripping leading `cd ... &&` prefixes and collapsing whitespace. The task key is the issue text; it is pinned by every compressor and therefore always correct.

Use probes. Identifiers are maximal matches of `[A-Za-z_][\w./-]{3,}[\w]` (names, dotted names, paths without the leading slash). A use probe is an identifier that occurs in the next action, is not a stop word (interface commands, Python keywords, common shell tools), occurs in some earlier turn of the history, and does not occur in the system prompt (which every compressor keeps). We bucket use probes by the number of steps since the identifier *last* occurred; "long-range" use probes are those whose last occurrence is more than two steps back (i.e. outside the most recent turns that every compressor keeps).

Copy probes. A copy probe is a normalized line of at least 12 characters of the next action that occurs as a line of an earlier non-system turn and not in the system prompt; it is available if the line occurs in the compressed context.

Decision points. For each trajectory we take up to six evenly spaced decision points (just before an agent turn) whose uncompressed history (excluding the system prompt) exceeds 4k tokens; a budget B is evaluated on the points whose history exceeds B. Every compressor is evaluated on the same points. Tokens are counted with a BPE tokenizer (the legacy Claude tokenizer shipped with version 0.28.0 of the MIT-licensed `anthropic` Python SDK); the system prompt, which every method keeps, is not charged to the budget.

C Baseline and SKC Settings

- **Recency window**: task pinned; most recent whole turns that fit; the last turn is truncated (head and tail) if it alone exceeds the budget.
- **Observation masking** (Lindenbauer et al., 2025): all agent turns kept; tool outputs other than the 10 most recent replaced by a placeholder; then recency window.
- **Truncate observations**: tool outputs other than the 2 most recent are cut to 200 tokens (one third head, two thirds tail); then recency window.
- **BM25 selection**: last 2 turns kept; other turns ranked by BM25 ($k_1 = 1.2, b = 0.75$) against the task and the latest agent turn and added greedily; original order preserved.
- **Random selection**: last 2 turns kept; other turns added in random order while they fit.
- **BM25 chunks**: the same 12-line chunks of older turns as SKC (no ledger, no deduplication), admitted greedily by BM25 score per token against the task and the latest agent turn; last 2 turns kept.
- **Self-information**: a model-free version of Selective Context (Li et al., 2023): lines of older turns are ranked by their mean self-information under a unigram model of the history and kept greedily; last 4 turns kept.
- **SKC-basic**: ledger with priority $p_k = w_{\text{type}}\tau/(\tau + \text{age}_k)$ ($w_{\text{edit}} = 4$, $w_{\text{cmd}} = 2$, $w_{\text{search}} = 1.5$, $w_{\text{view}} = 1$, $\tau = 20$ steps), value cap 300 tokens, $R = 4$ recent turns, remaining budget spent extending the raw window backwards and then on agent turns only.

- **SKC**: $R = 2$, chunks of at most 12 lines / 200 tokens, value cap $V = 300$, $\lambda = 0.3$, supersession-aware deduplication (lines of at least 6 normalized characters). Utility model: `HistGradientBoostingClassifier` (300 iterations, learning rate 0.08, 31 leaves, L2 1.0), trained on up to 8 decision points per training trajectory with negatives subsampled at 30% and re-weighted. Features: the 28 listed in Section 5 plus three added after a first evaluation and validated on the development split (BM25 against the latest agent turn alone, and the number of the item's lines that also occur in the latest agent and tool turns or in the last four turns). R, chunk size and λ were chosen on the development instances from $\{2, 4\} \times \{12, 24, 48\} \times \{0.1, 0.3\}$ (Section 6). A variant trained to predict the *number* of needed pieces per item (Poisson loss), which matches the knapsack objective more closely, was not better (within one point on every metric) and is not used.

In StateTrack the extractor reads the `key = value` lines that carry writes; on SWE-agent it is the structural parser above. Noisy extractors drop each write independently (deterministically by a hash of key and turn) with probability $1 - r$.

D Additional Results

Table 5: State probes, % correct at 8k, per LLM configuration (micro averages within each configuration). SKC and SKC-basic are best for every configuration.

	SWE-bench Lite (SWE-agent)					SWE-bench Verified						
	GPT-4	C3-Opus	GPT-4o	C3.7-S	C4-S	C4-S (OH)	GPT-5 (OH)	K2 (OH)	Qwen3-C (OH)	Devstral (OH)	LM-32B (SA)	K2 (SA)
Recency window	55.5	72.9	49.5	52.0	37.0	44.0	51.5	45.6	34.2	51.4	50.6	59.3
Observation masking	55.2	69.9	48.3	44.1	28.8	39.5	49.1	42.3	28.6	46.1	42.3	52.0
Truncate observations	58.1	73.4	55.2	43.1	41.2	40.8	51.4	40.3	32.8	48.3	38.2	54.8
BM25 (turns)	58.9	74.9	55.4	45.2	27.9	33.7	48.2	37.5	25.0	38.6	39.5	51.0
BM25 (chunks)	61.0	81.3	59.0	45.8	32.6	30.7	44.5	32.3	27.8	38.8	38.6	53.7
Self-information	46.1	72.2	44.7	30.8	21.0	21.5	30.2	23.6	22.8	23.7	27.5	42.1
Random selection	70.8	88.0	67.7	58.4	47.6	48.1	57.9	48.3	41.8	56.2	55.0	67.3
SKC-basic (ours)	73.4	87.3	78.5	**70.4**	**71.0**	**61.8**	**78.6**	**59.6**	**61.7**	**69.0**	**65.3**	**83.0**
SKC (ours)	**74.4**	**93.0**	**82.4**	69.2	66.9	55.8	71.6	53.3	55.4	63.4	61.5	82.8

Table 6: In-domain state probes at 8k by key family: % correct and % stale. Probe counts: edit 21.6k, view 29.4k, cmd 44.0k, search 7.9k, task 5.2k.

	Correct (%)					Stale (%)			
	task	edit	view	cmd	search	edit	view	cmd	search
Recency window	100	46.0	42.1	44.4	34.6	0.6	0.2	0.1	0.0
Observation masking	100	47.3	35.9	33.9	31.6	0.8	0.4	0.1	0.0
Truncate observations	100	51.9	25.5	50.2	63.1	1.4	1.4	0.7	0.2
BM25 (turns)	100	52.4	43.3	30.9	28.8	2.2	2.5	1.3	0.3
BM25 (chunks)	100	47.2	30.8	43.9	65.2	2.0	2.2	1.4	0.4
Self-information	100	27.3	33.7	25.8	31.1	1.1	2.0	0.8	0.1
Random selection	100	52.1	**56.2**	55.4	65.3	2.7	3.1	1.6	0.5
SKC-basic (ours)	100	**74.3**	45.9	**84.6**	88.3	0.3	0.4	0.1	0.0
SKC (ours)	100	72.5	50.6	79.5	**89.1**	0.3	0.3	0.1	0.0

File views are the one family where random selection keeps more: views are long (often a 100-line window), their ledger values are capped at 300 tokens, and the type prior ranks them last. Command outputs and search results, in contrast, are small records once the surrounding output is dropped.

Baselines that keep whole turns (window, masking, random) take 4–23 ms per call; chunk-level BM25 takes 133–349 ms, as it scores the same chunks as SKC. Window, masking and truncation leave part of the budget unused at larger budgets (e.g. masking uses 15.2k of 32k tokens) because they never split turns; SKC uses

Table 7: Compression and cost (in-domain). History: BPE tokens of the uncompressed history (excluding the fixed system prompt) at the evaluated decision points. Latency: wall-clock time of one compression call on one CPU core (Python), including utility-model inference.

Budget	Decision points	History (mean / median)	SKC tokens	Compression	SKC latency
4k	7,886	18.6k / 11.6k	3,946	4.7×	122 ms
8k	5,196	25.4k / 19.4k	7,701	3.3×	164 ms
16k	3,114	34.9k / 30.3k	14,768	2.4×	225 ms
32k	1,437	49.0k / 43.1k	28,734	1.7×	311 ms

90–99% of the budget; redundancy-aware selection leaves part of the 32k budget unused when every remaining item would only repeat emitted lines.

Table 8: How well item scores predict what the next action uses (held-out LLM and instances; averages over the ten folds).

Scorer	AUC	AP
SKC utility (gradient-boosted trees, 31 features)	**0.818**	**0.301**
Logistic regression (28 features)	0.806	0.282
BM25 relevance to task + latest agent turn	0.707	0.180
Number of later references	0.690	0.161
Recency	0.583	0.129

Table 9: Sensitivity of state probes to the evidence threshold θ (in-domain, 8k, % correct; SKC with the 28 v1 features). The ordering of methods is unchanged.

	$\theta = 0.6$	$\theta = 0.8$	$\theta = 1.0$
Recency window	58.3	55.5	53.2
Observation masking	54.3	50.8	47.9
BM25 (chunks)	62.2	56.6	51.9
Random selection	70.7	67.3	64.1
SKC-basic (ours)	**80.7**	**77.6**	**74.4**
SKC (ours)	79.8	74.9	70.2

E End-to-End Evaluation on ALFWorld

We play the 134 unseen-split games of ALFWorld (text version, TextWorld PDDL files) with `qwen3.8-flash` (thinking disabled, temperature 0, at most 50 steps). At every step the history is compressed to a budget of 300, 400, 600 or 1200 tokens (task and history; the fixed system prompt is not charged), measured on the rendered prompt. The task description alone takes roughly 100–200 tokens, so the two smallest budgets force real compression. SKC uses a domain extractor that writes the latest contents and open/closed state of each receptacle, the held object and cleaned/heated/cooled objects to the ledger; it compacts only once the history exceeds the budget and makes no LLM calls. The rolling summary folds the oldest steps into a running summary written by the same model (one extra call each time the budget overflows; 6.3 calls per game at 300 tokens). Success is paired per game; p-values are exact McNemar tests of SKC against each method.

Compression therefore costs success when the budget is tight, but the methods are indistinguishable from one another. SKC's ledger helps on the cooling tasks (81% vs. 57–67% at 300 tokens) but not on the others, and the per-type counts are small (17–34 games). SKC and the summary make more invalid commands than a window at the tightest budgets (2.8 and 4.6 vs. 1.9 per game). A plausible reason is that these games are short and the facts a window loses (which receptacles were visited and what they held) are cheap to rediscover, while the agent's failures come from planning rather than from missing state. We therefore make

Success (%)	300	400	600	1200
Window	72.4	77.6	83.6	85.8
Observation masking	70.1	76.1	83.6	88.8
LLM summary (rolling)	70.1	77.6	85.8	84.3
SKC	72.4	76.9	83.6	87.3
No compression	90.3 (95% CI 85.1–94.8)			

Table 10: ALFWorld success rate over 134 games (95% CIs are about ±7–8 points). No pairwise difference between compressors is significant (all $p \geq 0.45$, exact McNemar). Against no compression, every compressor is much worse at 300 tokens (SKC -17.9, $p < 0.001$) and 400 tokens (-13.4, $p = 0.001$), and the gap closes as the budget grows (-6.7 at 600, $p = 0.078$; -3.0 at 1200, $p = 0.48$).

no end-to-end claim: on short-horizon household tasks the state-keeping advantage measured by the probes does not translate into a measurable success-rate gain.

F StateTrack Details

The generator produces agent-like trajectories with three key families: *pinned* constraints stated in the task and occasionally revised by the user ($p = 0.004$ per key and step), *slow* facts rewritten rarely ($p = 0.03$), and *fast* status variables rewritten often ($p = 0.25$). Writes appear as `key = value` lines inside tool outputs (80%) or agent notes (20%), among log-like distractor lines (lognormal, mean 25 lines per output). Values are unique, so the ideal reader is exact. Unless stated otherwise we use 150 steps, 4 pinned, 12 slow and 6 fast keys, 30 seeds, and probe all keys at three checkpoints. The ledger-based compressor here is SKC-basic (there is no training corpus for a utility model); "+BM25" replaces its backward extension by BM25 selection.

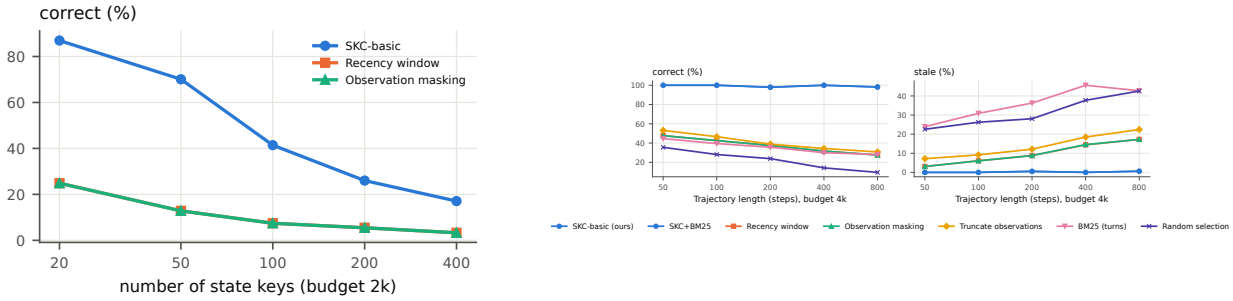


Figure 10: StateTrack. Left: retention as the number of keys grows at a fixed 2k budget (200 steps); full retention needs space linear in the number of live keys (Proposition 1): the ledger keeps 87% of 20 keys and 17% of 400, a window 25% and 3%. With uniform probe probabilities and equal record sizes, recency-only and FIFO priorities match the default to within one point (Proposition 5). Right: accuracy and staleness vs. trajectory length at 4k.